

ADLC for Salesforce

Agent Development Life Cycle
A Phase-Based Agent Architecture
with Dynamic Knowledge Retrieval

LKInsurance Project
April 2026 | Version 2.0

Table of Contents

- 1. Executive Summary
- 2. Design Principles
- 3. System Architecture (incl. Model Selection, Extended Thinking)
- 4. Phase Agents (Discovery, Design, Implementation, Testing, Deployment, Quick Fix)
- 5. Knowledge Layer (DX MCP, Local RAG, Prompt Caching, Citations)
- 6. Artifact Handoff System
- 7. Directory Structure
- 8. Cursor Hooks Configuration
- 9. MCP Configuration
- 10. Extending to New Salesforce Clouds
- 11. Context Budget Analysis
- 12. Phase Evaluation Framework (incl. Structured Output Schemas)
- 13. Implementation Roadmap
- 14. Retrofitting Existing Projects

1. Executive Summary

This proposal defines an AI-assisted development system for Salesforce projects built on two core ideas:

One agent per SDLC phase

Instead of loading every possible skill and rule into a single conversation, each phase of development -- Discovery, Design, Implementation, Testing, Deployment -- is handled by a dedicated AI agent with a focused context window. A human reviews and approves the output of each phase before the next begins. A lightweight Quick Fix agent handles trivial tasks that do not warrant the full pipeline.

Dynamic knowledge retrieval

Instead of manually curating knowledge bases per Salesforce cloud, the system uses two MCP servers -- the official Salesforce DX MCP Server for live org data and LWC knowledge, and a local RAG server backed by automatically downloaded Salesforce documentation for industry cloud knowledge. Adding support for a new Salesforce cloud takes minutes, not hours.

The system further optimizes by assigning model tiers per task -- frontier models (Opus, o3) for architecture and design reasoning, balanced models (Sonnet, GPT-4.1) for code generation, and fast models (Haiku, Gemini Flash) for test boilerplate and declarative configuration. A structured evaluation framework with automated checks and scored rubrics governs every phase gate, ensuring quality is measured consistently rather than left to ad-hoc review.

Claude API Platform Features

The architecture leverages six Claude-specific API features: Extended Thinking provides auditable reasoning traces for frontier-tier architecture decisions; Prompt Caching reduces rule-content token costs by ~60-70%; Message Batches enables parallel test generation at 50% cost reduction; Citations make every RAG-sourced claim traceable; Structured Outputs produce machine-readable evaluation artifacts for programmatic gate enforcement; and Cursor Hooks automate phase transitions, prerequisite validation, and metrics logging.

Together, these produce a system that is phase-disciplined, context-efficient, human-governed, cloud-extensible, cost-optimized, and production-focused.

2. Design Principles

Focused context over universal context. Each agent loads only the rules and knowledge relevant to its phase. This keeps the context window lean (30-65KB per agent) and reduces noise, contradictions, and hallucination risk.

Structural human-in-the-loop. Human review is not a rule the AI must remember to follow -- it is a physical gate between phases. The Implementation agent cannot run until a human has approved the Design artifact.

Artifacts as contracts. Phases communicate through structured markdown artifacts, not conversation

history. Each artifact is self-contained.

Two knowledge sources, clear boundaries. The Salesforce DX MCP Server handles live org operations and built-in LWC/SLDS knowledge. A local RAG MCP server handles industry cloud documentation. No overlap.

Cloud-extensible by configuration. Adding a new Salesforce cloud requires editing one JSON file and running one CLI command. No code changes, no account dependencies, no manual uploads.

Right model for the right task. Frontier models handle deep reasoning (design, architecture). Fast models handle well-constrained, pattern-heavy tasks (test generation, config). This reduces cost by 5-10x on high-volume phases without sacrificing quality on critical ones.

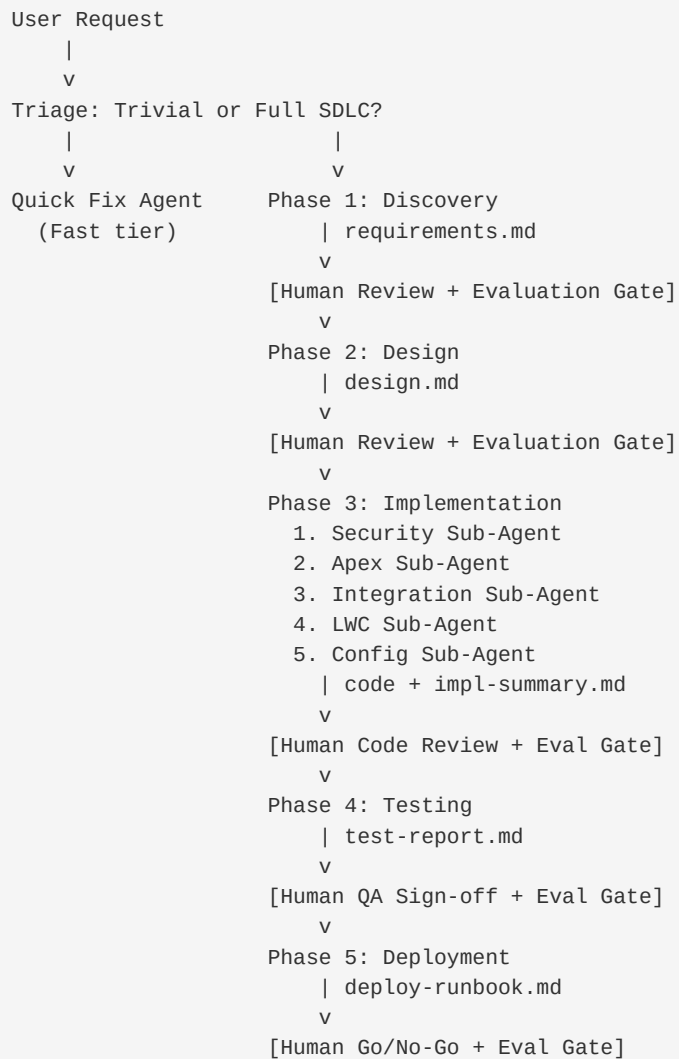
Measured quality at every gate. Each phase gate has automated checks (run by the agent) and a scored rubric (completed by the human reviewer). The evaluation record is appended to the artifact, creating an audit trail and feedback loop for refining model assignments.

Security as a first-class sub-agent. Security is not a checklist applied after implementation -- it is the first sub-agent dispatched. Permission sets, FLS, sharing rules, and encryption are defined before any code is written.

Machine-readable quality gates. Evaluation artifacts and security matrices are produced as structured JSON using Claude's structured outputs, then rendered as markdown for human readability. This enables automated gate enforcement, programmatic FLS validation, and trend analysis.

3. System Architecture

3.1 Overall Flow



3.2 Knowledge Architecture

Two MCP servers provide complementary knowledge:

MCP Server	What It Provides	Tools
Salesforce DX MCP (@salesforce/mcp)	Live org data, metadata, LWC/SLDS knowledge, code analysis, test execution	run_soql_query, deploy_metadata, guide_lwc_best_practices, run_code_analyzer, run_apex_test
Local RAG MCP (mcp-local-rag)	Industry cloud documentation, data models, business processes, regulations	query_documents, ingest_file, list_files

3.3 Knowledge Resolution

Priority	Source	What It Provides
1a	Salesforce DX MCP	Live org data, metadata, LWC/SLDS knowledge, code analysis, test execution
1b	Local RAG MCP	Industry cloud documentation, data models, business processes, regulations

2	Foundation rules	Salesforce platform fundamentals (naming, security, governor limits)
3	Phase-specific rules	Phase methodology and patterns (built into each agent's rule set)

3.4 Model Selection Strategy

Not every phase requires the same model. High-reasoning tasks need frontier models. High-volume, well-constrained tasks can use faster and cheaper models. Each phase agent specifies its model in its SKILL.md.

Model Tiers

Tier	Models	Characteristics
Frontier	Claude Opus, o3	Deep reasoning, multi-step planning, nuanced trade-offs. Highest cost/latency.
Balanced	Claude Sonnet, GPT-4.1	Strong code generation, good reasoning. ~3-5x cheaper than frontier.
Fast	Claude Haiku, GPT-4.1-mini, Gemini Flash	Pattern-following, template population. ~10-20x cheaper, ~3x faster.

Phase-to-Model Assignment

Phase / Sub-Agent	Tier	Rationale
Discovery	Frontier	Nuanced understanding of business requirements, feasibility judgment
Design	Frontier	Architecture decisions, Well-Architected assessment, security model design
Impl Coordinator	Frontier	Cross-sub-agent orchestration, dependency ordering
Impl: Security	Balanced	Permission set generation, FLS matrix, sharing rules
Impl: Apex	Balanced	Code generation with well-defined patterns (trigger framework, service layer)
Impl: Integration	Balanced	Callout patterns, Named Credentials, error handling
Impl: LWC	Balanced	Component generation with SLDS/accessibility patterns
Impl: Config	Fast	Declarative config is highly constrained (Flows, Validation Rules)
Testing	Fast + Balanced	Fast for batch test gen (Message Batches API); balanced for strategy
Deployment	Fast	Runbook generation, package.xml assembly are templated
Quick Fix	Fast	Trivial changes with narrow scope

Extended Thinking

Frontier-tier agents use Claude's extended thinking capability, which provides a dedicated reasoning scratchpad for multi-step trade-offs before producing the final artifact. Thinking blocks are saved as companion reasoning trace artifacts for reviewer audit.

Tier	Thinking Config	Budget
Frontier	thinking: {type: enabled, budget_tokens: 20000}	15,000-25,000 tokens
Balanced	Disabled	--
Fast	Disabled	--

Phase	Thinking Budget	What Gets Reasoned Through
-------	-----------------	----------------------------

Discovery	10-15K tokens	Synthesizing org data + cloud docs; feasibility assessment
Design	15-25K tokens	Config-First eval; Well-Architected trade-offs; security model
Impl Coordinator	5-10K tokens	Sub-agent dispatch ordering; dependency analysis

Escalation Pattern

If a fast-tier model produces output that fails the phase evaluation gate (Section 12), the phase is re-run with the next tier up: Fast -> Balanced -> Frontier. The evaluation artifact records which tier produced the passing output, providing data to refine tier assignments over time.

SKILL.md Model Field

Each agent's SKILL.md includes a model directive with optional thinking configuration. The Cursor Task tool reads this when spawning the agent.

```
---
model: opus
fallback_model: sonnet
thinking:
  enabled: true
  budget_tokens: 20000
---
```

4. Phase Agents

4.1 Phase 1: Discovery Agent

Model tier: Frontier (extended thinking: 10-15K budget)

Purpose: Understand the requirement, query the org, produce a structured requirements document.

Process

1. Ask clarifying questions (functional, non-functional, constraints)
2. Query org via DX MCP (run_soql_query) to discover existing objects, fields, automation
3. Query Local RAG for cloud-specific business process documentation (with citations)
4. Identify impacted objects and existing automation
5. Assess: trivially solvable with configuration? (early exit to Quick Fix)

Output: requirements.md

- Feature name, description, priority
- Functional requirements (numbered with unique IDs)
- Non-functional requirements (performance, security, accessibility)
- Impacted objects and fields (from org query)
- Existing automation found (from org query)
- Constraints, assumptions, acceptance criteria

- Sources (citations from RAG queries)

Reasoning trace: requirements-reasoning.md (saved thinking blocks for audit)

Human Checkpoint: Review requirements and optionally inspect reasoning trace. Approve or revise.

4.2 Phase 2: Design Agent

Model tier: Frontier (extended thinking: 15-25K budget)

Purpose: Produce a solution design based on approved requirements.

Process

1. Evaluate Configuration-First (Flow vs Apex vs Validation Rule vs Formula)
2. If declarative: document the configuration approach
3. If code needed: design data model, component architecture, integration points
4. Query Local RAG for industry cloud data models and regulations (with citations)
5. Query DX MCP for LWC best practices
6. Assess against Well-Architected pillars (TRUSTED, EASY, ADAPTABLE)
7. Identify which sub-agents are needed in Implementation

Output: design.md

- Approach decision (declarative vs code, with rationale)
- Data model changes (new objects/fields, ERD)
- Component architecture (Apex, LWC, Flows)
- Integration design (endpoints, auth, sync/async, error handling)
- Security model (CRUD/FLS plan, sharing model, encryption needs)
- Well-Architected assessment and risk assessment
- Sources (citations from RAG and DX MCP queries)

Reasoning trace: design-reasoning.md (thinking blocks showing architecture derivation)

Human Checkpoint: Review design and reasoning trace. Approve architecture.

4.3 Phase 3: Implementation Agent

Purpose: Build the solution per approved design. Coordinator dispatches to sub-agents.

Coordinator (Frontier tier, extended thinking: 5-10K budget)

- Reads requirements.md + design.md
- Dispatches sub-agents in dependency-aware order:
 - 1. Security -> 2. Apex -> 3. Integration -> 4. LWC -> 5. Config
- Validates integration between sub-agent outputs

Security Sub-Agent (Balanced tier)

Purpose: Define permission model before any code is written.

- Object-level, field-level, and record-level security

- Permission Set and Permission Set Group definitions
- Shield Platform Encryption for PII/PHI fields
- FLS matrix produced as structured JSON (see Section 12.8)

Apex Sub-Agent (Balanced tier)

- DX MCP: run_code_analyzer, scan_apex_class_for_antipatterns
- References security model: all data access uses WITH USER_MODE or stripInaccessible()
- Produces: Apex classes/triggers with naming, security enforcement, bulkification

Integration Sub-Agent (Balanced tier)

Purpose: Handle all external system integrations.

- Named Credentials, callout patterns, error handling, Platform Events, CDC
- Async patterns (Queueable callouts, chaining, idempotency)
- Produces: Callout classes, Event defs, error utilities, HttpCalloutMock

LWC Sub-Agent (Balanced tier)

- DX MCP: guide_lwc_development, explore_lbc_components, explore_slds_blueprints
- Produces: LWC components with SLDS styling, accessibility, FLS-aware rendering

Config Sub-Agent (Fast tier)

- DX MCP: retrieve_metadata (check existing flows)
- Produces: Flow definitions, validation rules, config instructions

Output: impl-summary.md

- Files created/modified with paths and responsibilities
- Integration points, security model, component dependencies

Human Checkpoint: Code review using Implementation Evaluation rubric (Section 12.4).

4.4 Phase 4: Testing Agent

Model tier: Balanced (strategy) + Fast (batch generation via Message Batches API)

Purpose: Produce test classes, test plans, and validate coverage.

Phase A: Test Strategy (balanced tier)

1. Analyze all Apex classes, LWC components, and integration endpoints
2. Create test plan (positive, negative, bulk, edge, security, integration)
3. Produce test manifest listing all test classes to generate

Phase B: Batch Generation (fast tier, Message Batches API)

4. Submit all test class generation requests as a single Claude Message Batch
5. Each request includes: source class + test data factory + testing rules
6. Batch completes asynchronously; 50% cost reduction vs sequential calls

Phase C: Validation

7. Execute all tests via DX MCP (run_apex_test)
8. Validate production readiness (validate_and_optimize, score_issues)
9. If failures, regenerate individual tests (not batch)

Output: test-report.md

- Test classes, test plan, coverage per class, production readiness score
- Batch generation metadata (batch ID, completion time, per-request status)

Human Checkpoint: QA sign-off using Testing Evaluation rubric (Section 12.5).

4.5 Phase 5: Deployment Agent

Model tier: Fast

Purpose: Prepare deployment package and runbook.

Output: deploy-runbook.md

- Deployment method, pre/post steps, commands, rollback plan, smoke tests
- Permission set and sharing rule deployment included in package

Human Checkpoint: Go/no-go using Deployment Evaluation rubric (Section 12.6).

4.6 Quick Fix Agent

Model tier: Fast

Purpose: Handle trivial tasks that do not warrant the full 5-phase pipeline.

Scope: Typo fixes, simple renames, single field additions, simple validation rules.

Guardrail: If non-trivial, redirect to Phase 1: Discovery.

5. Knowledge Layer

5.1 Salesforce DX MCP Server (Official)

The official Salesforce DX MCP Server (@salesforce/mcp), released as part of Salesforce Headless 360 (April 2026), provides 60+ tools.

Toolset	Key Tools	Used By
orgs	list_all_orgs, create_scratch_org	Discovery, Deployment
metadata	deploy_metadata, retrieve_metadata	Impl (all), Deployment
data	run_soql_query	Discovery, Impl, Security
lwc-experts	guide_lwc_best_practices, explore_lbc_components	Design, Impl (LWC)
code-analysis	run_code_analyzer, scan_apex_class	Impl (Apex), Testing
testing	run_apex_test, run_agent_test	Testing
experts-validation	validate_and_optimize, score_issues	Testing

devops	commit/promote_devops_center_work_item	Deployment
--------	--	------------

5.2 Local RAG MCP Server + Citations

The DX MCP does not cover industry cloud-specific knowledge. This gap is filled by mcp-local-rag (v0.12.0), a zero-setup local RAG server.

- Runs entirely locally (no API keys, no cloud dependency)
- Supports PDF, Markdown, TXT, DOCX
- Semantic search with keyword boosting

Citations integration: RAG results are passed to Claude as document content blocks with citations: {enabled: true}. Claude's response includes character-level citations back to source documents. Every claim about a cloud data model or regulation is traceable.

5.3 Documentation Registry and Fetcher

A JSON registry (sf-docs-registry.json) maps each cloud to its official documentation. A Python CLI (sf-docs-fetch.py) downloads documentation automatically.

5.4 Foundation Rules

Rule File	Content
naming-conventions.md	PascalCase objects, camelCase methods, trigger naming, LWC naming
security-baseline.md	CRUD/FLS enforcement, WITH USER_MODE, sharing rules, bind variabl
governor-limits.md	100 SOQL queries (sync), 10K DML rows, bulkification, async patterns

5.7 Prompt Caching Strategy

Every agent loads foundation rules (~15-20KB) plus phase-specific rules. Claude's prompt caching marks these as cache_control: {type: ephemeral}. The first agent pays a 25% write premium; subsequent agents read at 90% discount.

Agent	Foundation Rules	Phase Rules	Cache Benefit
Discovery	WRITE	WRITE (phase-1)	--
Design	READ	WRITE (phase-2)	foundation: 90% off
Impl Coordinator	READ	WRITE (phase-3)	foundation: 90% off
Impl: Security	READ	READ (phase-3)	both: 90% off
Impl: Apex	READ	READ (phase-3)	both: 90% off
Impl: Integration	READ	READ (phase-3)	both: 90% off
Impl: LWC	READ	READ (phase-3)	both: 90% off
Impl: Config	READ	READ (phase-3)	both: 90% off
Testing	READ	WRITE (phase-4)	foundation: 90% off
Deployment	READ	WRITE (phase-5)	foundation: 90% off

Net input token cost reduction on rule content: ~60-70%. Cache TTL is 5 minutes (effective within the Implementation phase's rapid sub-agent sequence). Across phases separated by human review, cache expires and is re-written.

6. Artifact Handoff System

Artifacts are the contract between phases. Each phase agent produces a structured markdown file that the next phase reads as input. Each artifact includes an Evaluation section appended by the reviewer. Cursor hooks (Section 8) enforce prerequisite validation.

```
Discovery --> requirements.md --> [Eval Gate] --> Design
Design    --> design.md        --> [Eval Gate] --> Implementation
Impl      --> impl-summary.md  --> [Eval Gate] --> Testing
Testing   --> test-report.md   --> [Eval Gate] --> Deployment
Deploy    --> deploy-runbook.md --> [Eval Gate] --> Done
```

7. Directory Structure

```
.cursor/
|-- rules/
|   |-- foundation/ (naming, security, governor-limits)
|   |-- phase-1-discovery.md .. phase-5-deployment.md
|   |-- quick-fix.md
|   +-- active-project-context.md
|
|-- skills/
|   |-- discovery/SKILL.md
|   |-- design/SKILL.md
|   |-- implementation/SKILL.md
|   |   |-- security/ apex/ integration/
|   |   |-- lwc/ config/
|   |-- testing/ deployment/ quick-fix/
|   +-- _shared/knowledge-query-pattern.md
|
|-- artifacts/templates/ (+ {feature-name}/)
|-- knowledge/ (registry + cloud docs)
|-- hooks.json
|-- hooks/ (validate-phase-gate.sh,
|           log-agent-metrics.sh,
|           validate-artifact-schema.sh)
+-- tools/ (sf-docs-fetch.py, etc.)
```

8. Cursor Hooks Configuration

Cursor hooks automate and enforce the phase pipeline. They run as spawned processes at defined stages of the agent loop, communicating via JSON over stdio.

Hook Events

Hook Event	Purpose
sessionStart	Verify prerequisite artifacts exist with APPROVED gate decision
afterFileEdit	Auto-validate artifact against JSON schema on save
subagentStart	Log sub-agent invocation with model tier for cost tracking
stop	Verify output artifact created; validate schema; log metrics

Phase Gate Enforcement

Phase Agent	Required Prerequisite
Design	requirements.md + evaluation.json with APPROVED
Implementation	design.md + evaluation.json with APPROVED
Testing	impl-summary.md + evaluation.json with APPROVED
Deployment	test-report.md + evaluation.json with APPROVED
Discovery / Quick Fix	No prerequisites

Metrics Logging

Every invocation is logged to metrics.jsonl (one JSON line per invocation) with: timestamp, agent name, model tier, thinking budget, input/output/thinking tokens, cache status, duration, and gate decision. This enables cost analysis, model tier optimization, and cache hit rate tracking.

9. MCP Configuration

```
{
  "mcpServers": {
    "Salesforce DX": {
      "command": "npx",
      "args": [
        "-y", "@salesforce/mcp",
        "--orgs", "DEFAULT_TARGET_ORG",
        "--toolsets", "orgs,metadata,data,users,
          lwc-experts,code-analysis,testing,
          experts-validation,devops",
        "--allow-non-ga-tools"
      ]
    },
    "sf-knowledge": {
      "command": "npx",
      "args": [
        "-y", "mcp-local-rag"
      ],
      "env": {
        "BASE_DIR": ".cursor/knowledge"
      }
    }
  }
}
```

10. Extending to New Salesforce Clouds

Adding support for a new cloud requires three steps:

1. Update sf-docs-registry.json with PDF slugs (~5 min)
2. Run `python .cursor/tools/sf-docs-fetch.py --cloud {name}` (~1 min)
3. Query immediately -- RAG server auto-indexes new docs

No code changes. No account dependencies. No manual uploads.

11. Context Budget Analysis

Agent	Tier	Thinking	Cache	Est. Context
Discovery	Frontier	10-15K	WRITE	~40KB
Design	Frontier	15-25K	READ fnd	~60KB
Impl Coord	Frontier	5-10K	READ fnd	~50KB

Impl: Security	Balanced	--	READ both	~45KB
Impl: Apex	Balanced	--	READ both	~60KB
Impl: Integ	Balanced	--	READ both	~55KB
Impl: LWC	Balanced	--	READ both	~55KB
Impl: Config	Fast	--	READ both	~35KB
Testing	Fast+Bal	--	READ fnd	~65KB
Deployment	Fast	--	READ fnd	~40KB
Quick Fix	Fast	--	WRITE	~30KB

Each sub-agent uses 30-65KB. Knowledge queries happen on-demand via MCP. Prompt caching reduces rule-content costs by ~60-70%. Extended thinking adds reasoning tokens for frontier-tier agents (billed separately from context window).

12. Phase Evaluation Framework

Each phase gate combines automated checks (run by the agent before human review) with a structured rubric (scored by the human reviewer). Evaluation artifacts are produced as structured JSON (Section 12.8) for programmatic gate enforcement via hooks.

Evaluation Structure

1. Automated checks -- agent runs before presenting output. Failures block handoff.
2. Rubric -- scored criteria, each rated Pass / Partial / Fail by human reviewer.
3. Gate rule -- minimum conditions to proceed to next phase.
4. Evaluation record -- structured JSON (evaluation.json) + markdown in artifact.

12.2 Discovery Evaluation

Rubric

Criterion	Pass	Fail
Completeness	All user scenarios covered	Major user flows missing
Clarity	Unambiguous, testable requirements	Vague or contradictory
Feasibility	Constraints and risks identified	No feasibility analysis
Traceability	Unique IDs, links to user story	No traceability
Org Awareness	Existing objects/automation documented	No org query performed
Reasoning Transparency	Thinking trace shows clear synthesis	No thinking trace

Gate: Zero Fail. Max 1 Partial.

12.3 Design Evaluation

Rubric

Criterion	Pass	Fail
Architecture Soundness	Well-Architected pillars scored with evidence	No assessment

Security Completeness	CRUD/FLS plan, sharing model, encryption	No security analysis
Scalability	Governor limit analysis, bulk data considered	No scalability thought
Maintainability	Separation of concerns, clear boundaries	Tightly coupled design
Integration Design	All external touchpoints with auth/error handling	Not analyzed
Reasoning Transparency	Thinking trace shows Config-First eval	No thinking trace

Gate: All 3 Well-Architected pillars scored. Security has CRUD/FLS + sharing. Zero Fail.

12.4 Implementation Evaluation

Rubric

Criterion	Pass	Fail
Code Quality	Clean, DRY, ApexDoc on public methods	Duplicated logic, no docs
Security: CRUD/FLS	Every data access enforces CRUD/FLS	Access without enforcement
Security: Integration	Named Credentials, no secrets in code	Hardcoded endpoints
Bulkification	Collections-based, no SOQL/DML in loops	SOQL or DML in loops
Integration Robustness	Error handling, retry, timeout	No error handling
Permission Model	Perm sets + FLS matrix + OWD documented	No perm metadata

Gate: Code analyzer passes. Zero anti-patterns. All security checks pass. Zero Fail.

12.5 Testing Evaluation

Rubric

Criterion	Pass	Fail
Coverage Depth	Meaningful assertions, bulk + negative	Line coverage only
Edge Cases	Null handling, governor boundary	No edge case testing
Security Tests	System.runAs() with restricted profiles	No security-context testing
Integration Tests	Mock all endpoints + error responses	No callout mocks
Regression Safety	Test data factory, independent tests	Hard-coded IDs

Gate: 75% coverage per class. All green. Readiness above threshold. Zero Fail.

12.6 Deployment Evaluation

Rubric

Criterion	Pass	Fail
Runbook Completeness	Pre/post/rollback steps	Missing rollback

Environment Coverage	Sandbox validation planned	No sandbox plan
Rollback Plan	Step-by-step, tested in sandbox	No rollback plan
Permission Deployment	Perm sets + sharing in package	Security metadata missing

Gate: Validation deploy succeeds. Rollback documented. Zero Fail.

12.7 Evaluation Record Format

Evaluation is produced as structured JSON (evaluation.json) and rendered as markdown:

```
## Evaluation
Evaluator: [Name] | Date: [Date] | Model: [Tier]
Automated Checks: [x] All passed
Rubric: Completeness=Pass, Clarity=Pass, ...
Gate Decision: APPROVED / REVISE / ESCALATE
Notes: [Free-form reviewer notes]
```

12.8 Structured Output Schemas

Evaluation artifacts and FLS matrices use Claude's structured outputs (output_config.format: {type: json_schema}) to guarantee valid, machine-parseable output.

Evaluation Schema (all phases)

```
{
  "phase": "discovery|design|impl|testing|deploy",
  "model_tier_used": "frontier|balanced|fast",
  "thinking_budget_used": 15000,
  "automated_checks": [
    {"check": "...", "passed": true, "notes": "..."}
  ],
  "rubric_scores": [
    {"criterion": "...", "score": "Pass|Partial|Fail"}
  ],
  "gate_decision": "APPROVED|REVISE|ESCALATE",
  "citations_count": 12
}
```

FLS Matrix Schema (Security Sub-Agent)

```
{
  "fields": [
    {
      "object_field": "Policy__c.Premium__c",
      "encrypted": false,
      "permission_sets": [
        {"name": "Agent", "access": "Read"},
        {"name": "Manager", "access": "Read/Edit"}
      ]
    }
  ]
}
```

JSON artifacts enable: automated gate enforcement via hooks, FLS validation by diffing against actual permission set metadata from DX MCP, and trend analysis across projects.

13. Implementation Roadmap

Phase A: Foundation (Week 1)

1. Create foundation/ rules (naming, security, governor limits)
2. Create artifact templates with evaluation sections and Sources sections
3. Build evaluation rubric templates with structured output JSON schemas
4. Configure Salesforce DX MCP Server
5. Authorize a Salesforce org
6. Implement Cursor hooks (hooks.json + hook scripts)

Phase B: Knowledge Layer (Week 1-2)

7. Create sf-docs-registry.json
8. Build sf-docs-fetch.py CLI tool
9. Download documentation for active cloud
10. Configure mcp-local-rag
11. Create _shared/knowledge-query-pattern.md with citations integration

Phase C: Phase Agents + Claude API Features (Week 2-3)

12. Build Discovery agent (frontier, extended thinking, citations, prompt caching)
13. Build Design agent (frontier, extended thinking, citations, prompt caching)
14. Build Implementation coordinator + sub-agents:
 - Security (balanced, structured output FLS JSON)
 - Apex (balanced), Integration (balanced, citations), LWC (balanced), Config (fast)
15. Build Testing agent (balanced strategy + fast batch generation via Message Batches)
16. Build Deployment agent (fast)
17. Build Quick Fix agent (fast)

Phase D: Validation (Week 3-4)

18. Run real feature through all 5 phases with evaluation gates and hooks
19. Validate extended thinking (reasoning traces saved, quality improvement)
20. Validate prompt caching (cache hits during Impl sub-agents, cost reduction)
21. Validate batch generation (parallel test gen, coverage met)
22. Validate citations (traceable source references in artifacts)
23. Validate hooks (phase gate blocks without APPROVED; metrics logged)
24. Validate structured outputs (JSON schemas, FLS diff against metadata)
25. Validate cloud extensibility (add second cloud)
26. Refine rules, rubrics, model assignments, thinking budgets, templates

14. Retrofitting Existing Projects

The pipeline described in Sections 4-6 assumes a greenfield flow. Real-world adoption requires plugging into projects that are already mid-flight -- requirements may exist in Jira, design decisions in Confluence, and code may already be partially written.

14.1 Mid-Pipeline Entry Points

The ADLC pipeline supports entry at any phase. The constraint: all prerequisite artifacts must exist and pass evaluation before a phase can start. For existing projects, this means generating artifacts retroactively from existing documentation.

Current State	Artifacts to Generate	Resume At
Pre-Discovery	None (run Discovery normally)	Phase 1
Post-Discovery	requirements.md	Phase 2: Design
Post-Design	requirements.md + design.md	Phase 3: Impl
Post-Implementation	All three + fls-matrix.json	Phase 4: Testing
Post-Testing	All four + test-report.md	Phase 5: Deploy

14.2 Retrofit Agent (Balanced tier)

A dedicated agent that translates existing project documentation (Jira stories, Confluence pages, Word/PDF docs) into ADLC-compliant artifacts. It is not Discovery or Design -- it is a translation and extraction agent.

Process

1. Ingest existing documentation into Local RAG via ingest_file
2. Query the org via DX MCP to discover current state (objects, fields, automation, code)
3. Cross-reference existing docs against org state
4. Generate the required artifact(s) using ADLC templates, with citations to source docs
5. Produce gap-analysis.md listing what's missing

Citations are enabled: every generated requirement or design decision cites back to the source Jira story or Confluence page, making the generated artifacts auditable.

14.3 Gap Analysis

The Retrofit Agent always produces a gap-analysis.md alongside generated artifacts, listing missing information across five categories:

- Requirement gaps: stories without acceptance criteria, missing NFRs
- Design gaps: no Config-First assessment, no Well-Architected review
- Security gaps: no sharing model, no FLS matrix, missing CRUD/FLS enforcement in code
- Integration gaps: endpoints without Named Credentials, no error handling
- Testing gaps: Apex classes with 0% coverage, no bulk or security tests

Each gap is classified as blocker (must fill before proceeding) or acceptable risk (proceed with gap)

documented). The human reviewer makes the final call.

14.4 Retroactive Evaluation

Generated artifacts go through the same evaluation framework (Section 12) with one additional gate decision option:

Gate Decision	Meaning
APPROVED	Artifact meets all criteria. Proceed.
APPROVED_WITH_GAPS	Minimum criteria met; gaps documented as risks. Proceed.
REVISE	Critical issues. Fill gaps and re-generate.
ESCALATE	Fundamental problems requiring stakeholder decision.

APPROVED_WITH_GAPS is only available for retroactively generated artifacts. When used, the gap-analysis.md travels with the artifacts as a risk register. Once all gaps are resolved, the project transitions to the standard pipeline.

14.5 Existing Code Onboarding

When entering at Post-Implementation (code already exists), the Retrofit Agent runs a reverse-engineering pass:

1. DX MCP: run_code_analyzer + scan_apex_class on all Apex classes
2. DX MCP: retrieve_metadata for permission sets, sharing rules, OWD, objects, flows
3. DX MCP: run_soql_query to map object relationships and record counts
4. Generate fls-matrix.json from actual deployed permission set metadata
5. Generate impl-summary.md from actual code + metadata
6. Flag: code quality issues, missing CRUD/FLS, hardcoded creds, untested classes

14.6 Hook Behavior for Retrofit

The sessionStart hook supports a retrofit mode flag. When set, the hook accepts APPROVED_WITH_GAPS as a valid gate decision (in addition to APPROVED) and logs the entry as a retrofit in metrics.jsonl for adoption tracking.

End of Document